成都站 铜牌冲刺板子 2026

先识别题型、写出证明，再按稳定编号查模板。所有正式代码都由仓库源码自动生成。

赛站纸质资料数量与规格：待赛站确认。25 页不是已核实的成都站限制。

统一提醒：多测新建容器；类型与区间以各模块契约为准；所有算术结果须处于声明范围。

思维卡片见第 2–3 页；排错与最后复习见第 4 页。

编号	算法与识别入口	页码
CO01	主程序骨架 Main	5
CO02	前缀、差分、离散化 Basics	6
CO03	答案二分、双指针、窗口 Search	8
DP01	0/1 与完全背包 Knapsack	9
DP02	线性 DP 与最长递增子序列 LIS	10
DP03	小规模状压最短路径 Hamilton DP	11
DS01	树状数组 Fenwick	12
DS02	单点线段树 Segment Tree	13
DS03	懒标记线段树 Lazy Segment Tree	14
DS04	单调栈与单调队列 Monotonic	16
G01	并查集 Disjoint Set Union	17
G02	BFS 最短路与路径恢复 BFS Path	18
G03	迭代 DFS 连通块与父亲 Iterative DFS Forest	19
G04	拓扑排序 Topological Sort	20
G05	简单无向基环图环恢复 Unicyclic Graph	21
M01	整数与位运算 Integer and Bits	23
M02	安全模运算与快速幂 Modular Power	24

思维卡片 从题意走到证明

构造 先证必要 再证足够

先枚举 n≤6 的小数据，记录可行性；从总和、奇偶、极值寻找必要条件。随后给出达到条件的构造，并逐条验证题目限制。找到一种答案不等于证明所有可行情形。

例：cf/construction/1.cpp 的回文构造；先传播固定镜像，再扣计数与处理中点，最后独立检查长度、数量和回文。代码记录不代表 AC。

操作 每一步改变了什么

把一次操作写成变量变化式。检查总和、差值、奇偶、模数、连通性。先证明操作次数下界，再考虑能否达到；不变量相同只是必要条件，可能仍不可达。

例：2025 成都 D，状态可重复出现，要记录访问状态和前驱；搜索终点后沿前驱恢复合法操作。

贪心 先找反例 再找交换

写清局部选择，枚举小数据寻找反例。尝试把某个最优方案的第一步换成自己的选择，证明可行且答案不变差。无法完成交换时不要把直觉当证明。

例：2025 成都 J 的分组调度与 C 的逆向排序；练习把剩余问题的结构写在纸上。

模拟 状态与更新次序

列出会变化的全部状态。按题面一次操作的真实先后更新，区分同时更新与依次更新；画出第 0 步、第一次、最后一次。先保证直接模拟正确，再证明跳步。

例：2024 成都 J 与 2025 成都 J，优先读清事件顺序，不因题短而省掉手算。

思维卡片 从暴力走到算法

区间操作 试着写差分

对闭区间 [l,r] 加 x，差分只改 d[l]+=x、d[r+1]-=x。观察正负差分如何配对。最优操作数必须有下界证明，不能把树状数组或线段树当成建模本身。

现场：画出 n=1、l=r、r=n 三个例子，明确是否需要 n+2 个位置。

图论 先辨认图的结构

先问：有向还是无向？是否连通？是否允许重边或自环？无向森林边数为 n-连通块数。简单连通基环图满足 m=n；去叶后得到环，再沿环恢复顺序。

现场：DFS 必须区分父边与已访问节点；路径恢复用前驱，不把遍历顺序当路径。

DP 保留影响未来的信息

从暴力决策树开始。两段历史若对所有未来选择的影响一样，就能合并成同一状态。先写状态含义、初值、合法转移和答案位置，再考虑滚动数组。

例：0/1 背包处理第 i 件物品；倒序容量保证每件只用一次。完全背包正序容量允许同一件继续使用。

复杂度 先用约束排除

n≈20 才考虑 2^n 状态；n≈2e5 通常需要近线性或 n log n。数值大不等于状态多，寻找周期、矩阵或二分前必须证明能跳过的过程。

例：2025 成都 B 是 max-plus 状态转移矩阵，不能直接套普通模数乘法矩阵。

现场排错与最后复习

交题前 90 秒

手算最小规模、全部相同、严格递增/递减、最大值、无解、重复值。多测至少连跑两组不同大小。核对输入格式、初值、输出换行、数组容量与时间复杂度。

读题时把区间约定写在代码第一行。此书数组与图优先 1-indexed 闭区间，字符串/状态等例外逐模块注明。

WA

对照变量的数学含义逐行检查。重点查 < 与 ≤、前缀 sum(l,r)=sum(r)-sum(l-1)、懒标记下传、负数取模、不可达状态参与转移、相等元素处理、排序比较器严格弱序。

先保存失败输入，再做小数据枚举与独立暴力；不要同时改算法和多个边界。

RE 与整数溢出

越界检查空容器、1-indexed 的 n+1、差分的 r+1；避免深树递归。乘法先提升：1LL*a*b 或 (__int128)a*b。INF 加边权前检查不可达，long long 也有上限。

__builtin_ctz(0) 与 __builtin_clz(0) 未定义；位移使用 1ULL，位数必须小于类型位宽。

TLE 与止损

区分死循环、重复入队和复杂度超限。二分每轮必须缩小区间；Dijkstra 忽略过期堆条目。一次调试围绕一个假设，15 分钟无新证据就换人复核或切题。

五小时：前 20 分钟并行读题与找稳题；中段持续汇总候选题与证明；最后 40 分钟优先已证思路、短修复和现有提交。按实际熟悉程度分工，不预设谁擅长什么。

赛前最后一轮

先读思维卡片和本页；手敲并查集、Fenwick、Lazy 调用各一遍；复盘一题自己的构造和图论失误；确认纸质资料规则、工具链和睡眠。不要在最后一晚从零学习重型算法。

板子帮助减少编码错误，不能替代性质、证明与团队时间分配，也不能保证获铜牌。

CO01 主程序骨架 Main

识别：每场比赛新建文件；单测、多测都从这份骨架开始。

约定：GNU++20；默认单测，只有题面给出测试组数时才取消 cin >> T 注释。

复杂度：骨架 O(T) 次调用；具体复杂度由 solve 决定。

调用：把题目逻辑写进 solve()；需要多测时启用 cin >> T。

易错：把单测题第一个数字误读为 T。；关闭同步后不要混用 scanf 与 cin。；多测数据、答案和容器应在 solve 内初始化。

例题：在 solve 内先读 n，再声明 vector<ll> a(n+1)并读入 a[1..n]。仓库实例：cf/date/7/7.25/1.cpp，main 读取 T 并逐组调用 solve。

源码 core/main.cpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// GNU++20；数组常用 vector<ll> a(n + 1)，有效范围 [1,n]。
// 和、乘积可能超过 ll 时，先转 __int128 再运算。
void solve() {
    // 在这里读入并处理一组数据；每组状态尽量使用局部变量。
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    // 多测题取消下一行注释；单测题保持 T=1。
    // cin >> T;
    while (T--) solve();
    return 0;
}
```

CO02 前缀、差分、离散化 Basics

识别：静态区间和用前缀；离线区间加用差分；值域太大先离散化。

约定：a[1..n]，{}或{0}为空；非空查询须 1<=l<=r<=n，l>r 为 0 或不修改。Difference 初值全 0；所有 ll 中间值须不溢出。rank 只查已存在值。

复杂度：前缀建表 O(n)、查询 O(1)；差分修改 O(1)、还原 O(n)；离散化 O(n log n)、rank O(log n)；空间 O(n)。

调用：PrefixSum p(a); p.sum(l,r); Difference d(n); d.add(l,r,x); auto b=d.values(); Compression c(a); c.rank(a[i]); cout<<to_string_i128(x);

易错：前缀差是 pre[r]-pre[l-1]。；离散化保顺序，不保原距离；values[rank]还原原值。；__int128 最小值不能直接取负；输出前必须转字符串。

例题：a={0,4,-2,4}：PrefixSum(a).sum(2,3)=2；Compression(a).rank(4)=2。实题：洛谷 P8218 求区间和（静态闭区间求和）：洛谷 P8218 。

源码 core/basics.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 数组 a[1..n]；{} 或 {0} 表示 n=0。所有 ll 中间结果须不溢出。
inline string to_string_i128(__int128 x) {
    bool neg = x < 0;
    // 无符号取反能处理 -2^127，禁止直接对最小有符号数取负。
    __uint128_t u = neg ? -(__uint128_t)x : (__uint128_t)x;
    string s;
    do { s += char('0' + u % 10); u /= 10; } while (u);
    if (neg) s += '-';
    reverse(s.begin(), s.end());
    return s;
}
struct PrefixSum {
    int n;
    vector<ll> pre;
    explicit PrefixSum(const vector<ll>& a)
        : n(max(0, int(a.size()) - 1)), pre(n + 1) {
        for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
    }
    ll sum(int l, int r) const {
        if (l > r) return 0; // 空区间；非空时 1<=l<=r<=n。
        assert(1 <= l && r <= n);
        return pre[r] - pre[l - 1];
    }
};
// 离线区间加；初值全 0。values() 得到 1 下标数组，可重复调用。
struct Difference {
    int n;
    vector<ll> d;
    explicit Difference(int size = 0) : n(size), d(size + 1) {
        assert(size >= 0);
    }
    void add(int l, int r, ll x) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        d[l] += x;
        if (r < n) d[r + 1] -= x;
    }
    vector<ll> values() const {
        vector<ll> a(n + 1);
        for (int i = 1; i <= n; ++i) a[i] = a[i - 1] + d[i];
        return a;
    }
};
```

CO02 前缀、差分、离散化 Basics 续 2

接上一页；请合并复制整个模块。源码 core/basics.hpp

```
// 离散化保序不保距离。rank(x) 只接收初始化时存在的值。
struct Compression {
    vector<ll> values; // values[rank] 为原值；values[0] 不使用。
    explicit Compression(const vector<ll>& a) {
        values.assign(a.begin() + min<size_t>(1, a.size()), a.end());
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        values.insert(values.begin(), 0);
    }
    int rank(ll x) const {
        auto it = lower_bound(values.begin() + 1, values.end(), x);
        assert(it != values.end() && *it == x);
        return int(it - values.begin());
    }
};
```

CO03 答案二分、双指针、窗口 Search

识别：答案满足单调判断时二分；有序数组找二元和用双指针；非负数组求最长受限和区间用窗口。

约定：二分范围为整数闭区间；first_true 要求假到真，last_true 要求真到假，无解/空区间为 nullopt。数组 1 下标；two_sum 需非降序，窗口元素须非负。

复杂度：二分 O(log 范围)次 check、O(1)空间；双指针和窗口 O(n)时间、O(1)额外空间。

调用：auto ans=first_true(lo,hi,check); if(ans) cout<<*ans; auto [i,j]=two_sum_sorted(a,target); int len=longest_sum_at_most(a,k);

易错：必须先证明 check 单调；check 自己的乘加仍可能溢出。；two_sum 返回不同下标 i<j；无解为{0,0}。；窗口含负数时不能套用；limit<0 返回 0。

例题：first_true(1,10,[](ll x){return x>=7;})得到 7；窗口{0,2,1,3}、limit=3 的最长长度为 2。仓库实例：cf/other/Hello_2018/D.cpp，按时间排序后 check(x)判断能否选 x 题，二分最大可行 x。

源码 core/search_window.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 整数闭区间 [lo,hi]；空区间或无解返回 nullopt。
// ok 确定且单调；check 内部的乘加仍需自行防溢出。
template<class F>
optional<ll> first_true(ll lo, ll hi, F ok) {
    if (lo > hi || !ok(hi)) return nullopt; // false...false,true...true
    while (lo < hi) {
        ll mid = midpoint(lo, hi); // 向 lo 取整，全 ll 范围不溢出。
        if (ok(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
template<class F>
optional<ll> last_true(ll lo, ll hi, F ok) {
    if (lo > hi || !ok(lo)) return nullopt; // true...true,false...false
    while (lo < hi) {
        ll mid = midpoint(hi, lo); // 向 hi 取整，避免死循环。
        if (ok(mid)) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
// a[1..n] 非降序，找不同位置 i<j 且 a[i]+a[j]=target；无解 {0,0}。
inline pair<int, int> two_sum_sorted(const vector<ll>& a, ll target) {
    int l = 1, r = max(0, int(a.size()) - 1);
    while (l < r) {
        __int128 s = (__int128)a[l] + a[r];
        if (s == target) return {l, r};
        if (s < target) ++l;
        else --r;
    }
    return {0, 0};
}
// a[1..n] 必须非负；返回和<=limit 的最长连续段长度，可为 0。
// 负数会破坏窗口单调性；{}、{0} 均返回 0。和用 i128 防溢出。
inline int longest_sum_at_most(const vector<ll>& a, ll limit) {
    if (limit < 0) return 0;
    int n = max(0, int(a.size()) - 1), l = 1, best = 0;
    __int128 sum = 0;
    for (int r = 1; r <= n; ++r) {
        assert(a[r] >= 0);
        sum += a[r];
        while (sum > limit) sum -= a[l++];
        best = max(best, r - l + 1);
    }
    return best;
}
```

DP01 0/1 与完全背包 Knapsack

识别：按物品选择、总重量受容量限制，价值最大化；能区分每件一次与无限次。

约定：w、v 从 1 开始，w[i]>0，cap>=0；价值和任意可达总价值绝对值<2^60。默认总重量不超过容量；exact=true 表示恰好重量，不可达为 KNAP_NEG。

复杂度：时间 O(n*cap)，空间 O(cap)。

调用：vector<int> w(n+1); vector<ll> v(n+1); 读入 1..n；auto f=knapsack01(w,v,m,true); 判断 f[m]==KNAP_NEG 后输出。无限次改调 knapsack_complete。

易错：0/1 容量倒序，完全背包容量正序。；恰好装满必须只初始化 dp[0]=0，其他为负无穷；不能把负无穷直接相加。；不支持零重量物品；完全背包零重正价值会使答案无界。

例题：原仓库 01bag.cpp：https://github.com/ball-castle/template/blob/main/01bag.cpp 。保留原正确转移，新增 64 位与恰好容量契约；不宣称在线 AC。

源码 dp/knapsack.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// w、v 为 1-indexed；w[i]>0，cap>=0，价值及可达总和绝对值<2^60。
// exact=false：总重量<=j；exact=true：总重量恰好为 j。
// 不可达为 KNAP_NEG；不能直接对不可达状态加价值。
inline constexpr ll KNAP_NEG = -(1LL << 60);
inline vector<ll> knapsack01(const vector<int>& w,
                             const vector<ll>& v, int cap,
                             bool exact = false) {
    assert(cap >= 0 && !w.empty() && w.size() == v.size());
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0);
        // 倒序：dp[j-w[i]] 仍属于上一轮，物品最多取一次。
        for (int j = cap; j >= w[i]; --j)
            if (dp[j - w[i]] != KNAP_NEG)
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp;
}

inline vector<ll> knapsack_complete(const vector<int>& w,
                                    const vector<ll>& v, int cap,
                                    bool exact = false) {
    assert(cap >= 0 && !w.empty() && w.size() == v.size());
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0);
        // 正序：dp[j-w[i]] 可以已经取过本物品，允许继续取。
        for (int j = w[i]; j <= cap; ++j)
            if (dp[j - w[i]] != KNAP_NEG)
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp;
}
// auto best = knapsack01(w, v, capacity, true);
// best[capacity]==KNAP_NEG 表示不能恰好装满；否则就是最大价值。
```

DP02 线性 DP 与最长递增子序列 LIS

识别：寻找保持原顺序的最长严格递增或非递减子序列；先用朴素状态理解，再用二分优化。

约定：a 从 1 开始，空序列传 {0}；strict=true 严格递增，false 非递减。只返回长度，不恢复实际序列。

复杂度：lis_quadratic：O(n^2) 时间；lis_length：O(n log n) 时间；空间均 O(n)。

调用：vector<ll> a(n+1); 读入 a[1..n]；int ans=lis_length(a); 初学调试可用 lis_quadratic(a)对照。

易错：严格递增对应 lower_bound；重复值可接在后面时用 upper_bound。；tails 表示各长度的最小末尾，最终 tails 通常不是原数组的一个子序列。

例题：原仓库线性 dp.cpp：https://github.com/ball-castle/template/blob/main/%E7%BA%BF%E6%80%A7dp.cpp 。保留原 O(n^2) 状态，增加二分版。

源码 dp/lis.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// a 为 1-indexed；仅求长度。strict=true 严格递增，否则非递减。
// 最直观状态：dp[i] 为以 a[i] 结尾的最优长度，O(n^2)。
inline int lis_quadratic(const vector<ll>& a, bool strict = true) {
    assert(!a.empty());
    int n = int(a.size()) - 1, ans = 0;
    vector<int> dp(n + 1, 1);
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j < i; ++j) {
            bool can = strict ? a[j] < a[i] : a[j] <= a[i];
            if (can) dp[i] = max(dp[i], dp[j] + 1);
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}

// tails[k]：长度 k+1 的序列中，最小的末尾值；它不是答案序列。
// 严格递增用 lower_bound，非递减用 upper_bound；O(n log n)。
inline int lis_length(const vector<ll>& a, bool strict = true) {
    assert(!a.empty());
    vector<ll> tails;
    for (int i = 1; i < int(a.size()); ++i) {
        auto it = strict ? lower_bound(tails.begin(), tails.end(), a[i])
                         : upper_bound(tails.begin(), tails.end(), a[i]);
        if (it == tails.end()) tails.push_back(a[i]);
        else *it = a[i];
    }
    return int(tails.size());
}
// vector<ll> a(n+1); 读入 a[1..n]; cout << lis_length(a);
```

DP03 小规模状压最短路径 Hamilton DP

识别：n 很小，需要每个点恰好访问一次；用 mask 记录已选集合，末尾点作为第二维。

约定：w 为 1-indexed 有向图方阵，缺边为 HAMILTON_INF；从 1 到 n，每点恰好一次。0<=n<=18，n=0/1 约定为 0；允许负边，任意路径总权值在(-2^60,2^60)。

复杂度：时间 O(n^2*2^n)，空间 O(n*2^n)；n=18 约 36MiB。

调用：vector<vector<ll>> w(n+1,vector<ll>(n+1,HAMILTON_INF)); 填边；ll ans=hamilton_path(w); ans==HAMILTON_INF 表示无解。

易错：点 i 对应第 i-1 位，状态里的 u 是 0-indexed。；最短路、方案计数、是否可行的转移不同，不能原封不动混用。；终点固定为 n；任意终点需对最终 mask 的末尾状态取最小值。

例题：相关变体 CSES Hamiltonian Flights：https://cses.fi/problemset/task/1690 。原题计数且 n<=20，需把 min 转移改成求和并重新评估复杂度；当前模板是最短路，未在线提交。

源码 dp/hamilton_path.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// w 为 1-indexed 方阵；有向图，缺边用 HAMILTON_INF。
// 从 1 出发，到 n 结束，每个点恰好一次。n<=18；n=0/1 约定答案 0。
// 可以有负边；任意路径的权值总和须在 (-2^60,2^60) 内。
inline constexpr ll HAMILTON_INF = 1LL << 60;
inline ll hamilton_path(const vector<vector<ll>>& w) {
    assert(!w.empty());
    int n = int(w.size()) - 1;
    assert(n <= 18);
    for (const auto& row : w) assert(row.size() == w.size());
    if (n <= 1) return 0;
    int states = 1 << n;
    // 点 i 对应第 i-1 位；dp[mask*n+u] 中 u 是 0-indexed。
    // 状态表示已经访问 mask、当前停在 u+1 的最短路。
    vector<ll> dp(size_t(states) * n, HAMILTON_INF);
    dp[n] = 0;  // mask=1，停在点 1。
    for (int mask = 1; mask < states; mask += 2) {
        for (int u = 0; u < n; ++u) {
            ll cur = dp[size_t(mask) * n + u];
            if (cur == HAMILTON_INF) continue;
            for (int v = 0; v < n; ++v) {
                if ((mask >> v & 1) || w[u + 1][v + 1] == HAMILTON_INF)
                    continue;
                int next = mask | (1 << v);
                ll& dst = dp[size_t(next) * n + v];
                dst = min(dst, cur + w[u + 1][v + 1]);
            }
        }
    }
    return dp[size_t(states - 1) * n + n - 1];
}
// O(n^2*2^n) 时间、O(n*2^n) 空间；n=18 的 dp 约 36MiB。
// 返回 HAMILTON_INF 表示不存在；求任意终点则改取最后一行最小值。
```

DS01 树状数组 Fenwick

识别：单点加、前缀和、区间和；频次累计、逆序对可先离散化。

约定：n>=0，1 下标；add 位置须 1..n，sum 允许 0；range_sum(l,r)的 l>r 返回 0。树节点、前缀、答案及中间值均须在 ll 范围。

复杂度：单次增量/查询 O(log n)，空间 O(n)；逐点初始化 O(n log n)。

调用：Fenwick f(n); for(int i=1;i<=n;++i) f.add(i,a[i]); f.add(p,delta); ll ans=f.range_sum(l,r);

易错：add(0,x)非法，旧写法会死循环。；这里是增量 add；改成新值要传 new-old。；区间最小值不能照搬两个前缀相减。

例题：由旧 Fenwick_tree.cpp 提炼；修复 rangeSum 与 query 接口不一致。实题：洛谷 P3374 树状数组 1，单点加与闭区间求和：洛谷 P3374 。

源码 data_structure/fenwick.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 单点加、区间和。n>=0，1 下标；所有树节点/前缀/答案须在 ll 范围。
// 适用于可相减的前缀信息，不能直接把求和改成区间最小值。
struct Fenwick {
    int n;
    vector<ll> bit;
    explicit Fenwick(int size = 0) : n(size), bit(size + 1) {
        assert(size >= 0);
    }
    void add(int p, ll x) {
        assert(1 <= p && p <= n); // p=0 会使 lowbit 循环无法前进。
        while (p <= n) {
            bit[p] += x;
            int step = p & -p;
            if (step > n - p) break; // 避免索引加法越过 int 上界。
            p += step;
        }
    }
    ll sum(int p) const { // [1,p]，允许 p=0。
        assert(0 <= p && p <= n);
        ll ans = 0;
        for (; p; p -= p & -p) ans += bit[p];
        return ans;
    }
    ll range_sum(int l, int r) const { // [l,r]；空区间返回 0。
        if (l > r) return 0;
        assert(1 <= l && r <= n);
        return sum(r) - sum(l - 1);
    }
};
```

DS02 单点线段树 Segment Tree

识别：单点赋值、区间求和；需要以后扩展节点信息时使用。

约定：a[1..n]；{}或{0}为空树；set 位置须 1..n，非空 sum 须 1<=l<=r<=n，空区间返回 0。n<=INT_MAX/4；区间和及中间值须在 ll 范围。

复杂度：建树 O(n)；赋值/查询 O(log n)；空间 O(n)。

调用：SegmentTree seg(a); seg.set(p,x); ll ans=seg.sum(l,r);

易错：set(p,x)是赋值，不是加 x。；修改返回时必须 pull；答案与节点都用 ll。；空树只允许空查询，不允许 set。

例题：保留旧 segmen tree1.cpp 的递归建树/合并方法。仓库实例：cf/segment_tree_cf_edu/1.cpp，单点赋值、区间和；输入 0 下标[l,r)转成内部 1 下标[l+1,r]。

源码 data_structure/segment_tree.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 单点赋值、区间求和；a[1..n]，{} 或 {0} 表示空树。
// 任意区间和及中间值须在 ll 范围；n<=INT_MAX/4 且内存足够。
struct SegmentTree {
    int n;
    vector<ll> tr;
    explicit SegmentTree(const vector<ll>& a)
        : n(max(0, int(a.size()) - 1)), tr(4ULL * max(1, n)) {
        if (n) build(1, 1, n, a);
    }
    void set(int pos, ll x) { // a[pos]=x；不是单点加。
        assert(1 <= pos && pos <= n);
        update(1, 1, n, pos, x);
    }
    ll sum(int l, int r) const {
        if (l > r) return 0;
        assert(1 <= l && r <= n);
        return query(1, 1, n, l, r);
    }
private:
    void pull(int p) { tr[p] = tr[p * 2] + tr[p * 2 + 1]; }
    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) { tr[p] = a[l]; return; }
        int m = l + (r - l) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }
    void update(int p, int l, int r, int pos, ll x) {
        if (l == r) { tr[p] = x; return; }
        int m = l + (r - l) / 2;
        if (pos <= m) update(p * 2, l, m, pos, x);
        else update(p * 2 + 1, m + 1, r, pos, x);
        pull(p);
    }
    ll query(int p, int l, int r, int L, int R) const {
        if (L <= l && r <= R) return tr[p];
        int m = l + (r - l) / 2;
        ll ans = 0;
        if (L <= m) ans += query(p * 2, l, m, L, R);
        if (R > m) ans += query(p * 2 + 1, m + 1, r, L, R);
        return ans;
    }
};
```

DS03 懒标记线段树 Lazy Segment Tree

识别：在线区间加、区间和；更新整段后仍需查询任意子段。

约定：a[1..n]；{}或{0}为空树；l>r 时 add 无操作、sum 为 0，非空范围须合法。n<=INT_MAX/4；节点和、累积 tag、长度*x 及中间值均须在 ll 范围。

复杂度：建树 O(n)；区间加/求和 O(log n)；空间 O(n)。

调用：LazySegmentTree seg(a); seg.add(l,r,delta); ll ans=seg.sum(l,r);

易错：tag 表示已计入本节点、尚未传给子节点的增量。；查询部分覆盖时也必须 push；完全覆盖直接返回。；本模板仅有加法标记，不可直接混入赋值或乘法。

例题：旧 Segment tree2.cpp 最小反例：[0,0]全段+1 后查[1,1]，旧值 0、正确值 1。实题：洛谷 P3372 线段树 1，区间加与区间和：洛谷 P3372 。

源码 data_structure/lazy_segment_tree.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 区间加、区间和；a[1..n]，{} 或 {0} 为空；n<=INT_MAX/4。
// 节点和、标记累加、长度*x 及所有中间值必须在 ll 范围。
struct LazySegmentTree {
    int n;
    vector<ll> tr, tag;
    explicit LazySegmentTree(const vector<ll>& a)
        : n(max(0, int(a.size()) - 1)), tr(4ULL * max(1, n)), tag(tr.size()) {
        if (n) build(1, 1, n, a);
    }
    void add(int l, int r, ll x) {
        if (l > r) return; // 空区间不修改，包括 n=0。
        assert(1 <= l && r <= n);
        modify(1, 1, n, l, r, x);
    }
    ll sum(int l, int r) {
        if (l > r) return 0;
        assert(1 <= l && r <= n);
        return query(1, 1, n, l, r);
    }
private:
    void pull(int p) { tr[p] = tr[p * 2] + tr[p * 2 + 1]; }
    void apply(int p, int l, int r, ll x) {
        tr[p] += (r - l + 1LL) * x;
        tag[p] += x;
    }
    void push(int p, int l, int r) {
        if (!tag[p] || l == r) return;
        int m = l + (r - l) / 2;
        apply(p * 2, l, m, tag[p]);
        apply(p * 2 + 1, m + 1, r, tag[p]);
        tag[p] = 0;
    }
    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) { tr[p] = a[l]; return; }
        int m = l + (r - l) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }
    void modify(int p, int l, int r, int L, int R, ll x) {
        if (L <= l && r <= R) { apply(p, l, r, x); return; }
        push(p, l, r);
        int m = l + (r - l) / 2;
        if (L <= m) modify(p * 2, l, m, L, R, x);
        if (R > m) modify(p * 2 + 1, m + 1, r, L, R, x);
        pull(p);
    }
```

DS03 懒标记线段树 Lazy Segment Tree 续 2

接上一页；请合并复制整个模块。源码 data_structure/lazy_segment_tree.hpp

```
    ll query(int p, int l, int r, int L, int R) {
        if (L <= l && r <= R) return tr[p];
        push(p, l, r); // 查询也必须下传，否则子节点仍是旧值。
        int m = l + (r - l) / 2;
        ll ans = 0;
        if (L <= m) ans += query(p * 2, l, m, L, R);
        if (R > m) ans += query(p * 2 + 1, m + 1, r, L, R);
        return ans;
    }
};
```

DS04 单调栈与单调队列 Monotonic

识别：找每个位置左侧最近更小元素；固定窗口查询最小值。

约定：输入和输出均 1 下标；{}或{0}为空。previous_less 无解下标为 0；window_min 返回 ans[l]=min(a[l..l+k-1])，k<=0 或 k>n 返回{0}。

复杂度：均 O(n)时间，每个下标至多进出一次；额外空间 O(n)。

调用：auto left=previous_less(a); auto mn=window_min(a,k); // mn[l]对应窗口左端 l

易错：严格更小必须弹出>=当前值，重复值不能留下。；队列保存下标，才能删除离开窗口的元素。；窗口最小值改最大值时将队尾比较>=改为<=。

例题：a={0,2,2,1,3}的左侧严格更小下标为{0,0,0,0,3}；a={0,4,2,2,5},k=2 的最小值为{0,2,2,2}。实题：洛谷 P1886 滑动窗口，求固定窗口最小/最大值：洛谷 P1886 。

源码 data_structure/monotonic.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 输入 a[1..n]；{}、{0} 均为空数组；返回值也保留 0 号占位。
// 每个位置左侧最近的严格更小值的下标；不存在为 0。
inline vector<int> previous_less(const vector<ll>& a) {
    int n = max(0, int(a.size()) - 1);
    vector<int> ans(n + 1), st;
    for (int i = 1; i <= n; ++i) {
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        ans[i] = st.empty() ? 0 : st.back();
        st.push_back(i);
    }
    return ans;
}
// 固定长度 k 的窗口最小值；ans[l]=min(a[l..l+k-1])。
// k<=0 或 k>n 返回 {0}。相等时保留较新的下标。
inline vector<ll> window_min(const vector<ll>& a, int k) {
    int n = max(0, int(a.size()) - 1);
    vector<ll> ans(1);
    if (k <= 0 || k > n) return ans;
    deque<int> q;
    for (int i = 1; i <= n; ++i) {
        while (!q.empty() && q.front() <= i - k) q.pop_front();
        while (!q.empty() && a[q.back()] >= a[i]) q.pop_back();
        q.push_back(i);
        if (i >= k) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

G01 并查集 Disjoint Set Union

识别：只增加连边、判断同集合、维护连通块大小；Kruskal 的基础。

约定：n>=0，点编号 [1,n]；unite 返回是否真的合并；components 是当前块数；size 返回所在块大小。

复杂度：初始化 O(n)，路径减半与按大小合并的操作均摊 O(alpha(n))，空间 O(n)。

调用：cp::DSU d(n); d.unite(u,v); bool ok=d.same(u,v); int cnt=d.size(u);

易错：合并必须先找根；sz 只有根位置有效。；普通 DSU 不支持删边和回滚；多测重新构造。；下标 0 只作预留，不能作为真实点调用。

例题：仓库实用记录：cf/3/1109/3.cpp ，通过偏移边合并后判断置换可达性；已读源码，AC 状态未核验。

源码 graph/dsu.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 点编号 [1,n]；路径减半 + 按集合大小合并。
struct DSU {
    vector<int> p, sz;
    int components;
    explicit DSU(int n) : p(n + 1), sz(n + 1, 1), components(n) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        while (p[x] != x) {
            p[x] = p[p[x]];
            x = p[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b]; --components;
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return sz[find(x)]; }
};
} // namespace cp
```

G02 BFS 最短路与路径恢复 BFS Path

识别：无权图最短距离、最少操作次数、输出一条最短路径。

约定：g 长度 n+1，点 [1,n]；边端点必须合法；有向/无向均可，重边自环可用。dist=-1 表示不可达；path(t) 返回起点到 t（含端点），非法或不可达目标返回空。

复杂度：BFS O(n+m)，路径恢复 O(路径长度)，空间 O(n)。

调用：vector<vector<int>> g(n+1); g[u].push_back(v); auto r=cp::bfs(g,s); auto p=r.path(t);

易错：入队立即标记，不能等出队再标记。；仅适用单位边权；0/1 权用 0-1 BFS，非负任意权用 Dijkstra。；无向边必须双向加入；多测传入新图。

例题：来源起点：https://github.com/ball-castle/template/blob/main/bfs.cpp 。新测试中孤立点返回空路径，起点到自身返回单点路径；不声明旧源码 AC。

源码 graph/bfs_path.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// g.size()==n+1；点 [1,n]；有向/无向均可，边权均视为 1。
struct BFSResult {
    vector<int> dist, parent; // 不可达 dist=-1；起点 parent=0
    vector<int> path(int t) const {
        if (t < 1 || t >= (int)dist.size() || dist[t] < 0) return {};
        vector<int> ans;
        for (; t; t = parent[t]) ans.push_back(t);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
inline BFSResult bfs(const vector<vector<int>>& g, int s) {
    int n = (int)g.size() - 1;
    BFSResult r{vector<int>(n + 1, -1), vector<int>(n + 1)};
    if (s < 1 || s > n) return r;
    queue<int> q; q.push(s); r.dist[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (r.dist[v] == -1) {
            r.dist[v] = r.dist[u] + 1;
            r.parent[v] = u;
            q.push(v);
        }
    }
    return r;
}
} // namespace cp
```

G03 迭代 DFS 连通块与父亲 Iterative DFS Forest

识别：无向图连通块标号、父亲树、需要避开递归栈溢出的深图遍历。

约定：g 长度 n+1，合法点 [1,n]，无向边双向加入；允许自环重边。comp 从 1 开始，根 parent=0，order 是发现顺序。

复杂度：O(n+m) 时间，O(n) 额外空间；显式栈不占递归调用栈。

调用：auto r=cp::dfs_forest(g); int c=r.components; int p=r.parent[u]; bool same=r.comp[u]==r.comp[v];

易错：有向图调用得到的是搜索森林，不是强连通分量。；本模板不输出出栈时间；子树闭区间请用 Euler 模板。；根可能不止一个，处理整张图时不能只从 1 出发。

例题：旧版来源：https://github.com/ball-castle/template/blob/main/dfs.cpp 。新测试覆盖断开的图、自环重边及 200000 点链。

源码 graph/dfs_forest.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 无向图，点 [1,n]；显式栈模拟 DFS，重边/自环均可。
struct DFSForest {
    vector<int> parent, comp, order;
    int components = 0; // comp 从 1 开始；每个根的 parent=0
};
inline DFSForest dfs_forest(const vector<vector<int>>& g) {
    int n = (int)g.size() - 1;
    DFSForest r;
    r.parent.resize(n + 1); r.comp.resize(n + 1);
    vector<pair<int, int>> st; // (当前点，下一条邻边下标)
    for (int s = 1; s <= n; ++s) if (!r.comp[s]) {
        r.comp[s] = ++r.components;
        r.order.push_back(s); st.push_back({s, 0});
        while (!st.empty()) {
            auto& [u, i] = st.back();
            if (i == (int)g[u].size()) { st.pop_back(); continue; }
            int v = g[u][i++];
            if (r.comp[v]) continue;
            r.comp[v] = r.components; r.parent[v] = u;
            r.order.push_back(v); st.push_back({v, 0});
        }
    }
    return r; // order 为入栈顺序，父亲先于儿子
}
} // namespace cp
```

G04 拓扑排序 Topological Sort

识别：DAG 处理顺序、依赖任务、DAG DP、有向环检测。

约定：n>=1，g 长度 n+1，点 [1,n]，有向边；返回全部 n 点的一个合法顺序，有环返回空。允许重边，自环判为环。

复杂度：O(n+m) 时间，O(n) 额外空间。

调用：auto order=cp::topological_sort(g); if(order.empty()) { /* 有环 */ }

易错：重边也要各自计入入度和扣减。；不要把无向边双向加入后直接求拓扑。；普通队列不保证字典序最小；需要时换小根堆。

例题：仓库实例：cf/1704E.cpp，使用拓扑序逆序做 DAG DP；已读源码，AC 状态未核验。

源码 graph/topological_sort.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 有向图，点 [1,n]，n>=1；返回空表示有环；重边计入入度。
inline vector<int> topological_sort(const vector<vector<int>>& g) {
    int n = (int)g.size() - 1;
    vector<int> in(n + 1), order;
    for (int u = 1; u <= n; ++u)
        for (int v : g[u]) ++in[v];
    queue<int> q;
    for (int u = 1; u <= n; ++u) if (!in[u]) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--in[v] == 0) q.push(v);
    }
    if ((int)order.size() != n) return {};
    return order;
}
} // namespace cp
```

G05 简单无向基环图环恢复 Unicyclic Graph

识别：连通简单无向图恰有一个环，找环点、按环顺序做双指针或 DP。

约定：点 [1,n]，每条无向边输入一次；检查 n>=3、边数=n、合法端点、无自环重边与连通性。无效时 ok=false 且 cycle 为空；成功时 cycle 不重复首点，末尾与首点有边。

复杂度：含 set 重边验证 O(n log n) 时间，O(n) 空间；剥叶与环恢复本身 O(n)。

调用：vector<pair<int,int>> edges={{1,2},{2,3},{3,1}}; auto r=cp::unicyclic_graph(3,edges); if(r.ok) for(int u:r.cycle) cout<<u<<' ';

易错：本模板明确拒绝自环、平行边形成的 1/2 长度环。；只有 m=n 不够，还必须连通。；叶子剥除后只剩环点，但仍需恢复相邻顺序。；cycle 下标为 0..长度-1，点编号仍为 1..n。

例题：手算例：边 (1,2),(2,3),(3,1),(3,4)，返回三角环 [1,2,3] 或反向；(1,2) 重复两次的图拒绝。

源码 graph/unicyclic_graph.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct Unicycle {
    bool ok = false;
    vector<int> cycle; // 环顺序，不重复首点；末点与首点相邻
};
// 连通简单无向基环图，点 [1,n]；每条无向边只输入一次。
// 非 n 条边、重边、自环、越界、不连通均返回 {false,{}}。
inline Unicycle unicyclic_graph(int n, const vector<pair<int, int>>& edges) {
    if (n < 3 || (int)edges.size() != n) return {};
    vector<vector<int>> g(n + 1);
    vector<int> deg(n + 1), seen(n + 1);
    set<pair<int, int>> used;
    for (auto [u, v] : edges) {
        if (u < 1 || u > n || v < 1 || v > n || u == v) return {};
        if (u > v) swap(u, v);
        if (!used.insert({u, v}).second) return {};
        g[u].push_back(v); g[v].push_back(u);
        ++deg[u]; ++deg[v];
    }
    queue<int> q; q.push(1); seen[1] = 1;
    int count = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop(); ++count;
        for (int v : g[u]) if (!seen[v]) seen[v] = 1, q.push(v);
    }
    if (count != n) return {};
    for (int u = 1; u <= n; ++u) if (deg[u] == 1) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); deg[u] = 0;
        for (int v : g[u]) if (deg[v] && --deg[v] == 1) q.push(v);
    }
    int start = 0, length = 0;
    for (int u = 1; u <= n; ++u) if (deg[u]) {
        if (deg[u] != 2) return {};
        if (!start) start = u;
        ++length;
    }
    if (length < 3) return {};
```

G05 简单无向基环图环恢复 Unicyclic Graph 续 2

接上一页；请合并复制整个模块。源码 graph/unicyclic_graph.hpp

```
    Unicycle r; int prev = 0, u = start;
    for (int i = 0; i < length; ++i) {
        r.cycle.push_back(u);
        int next = 0;
        for (int v : g[u]) if (deg[v] && v != prev) { next = v; break; }
        if (!next) return {};
        prev = u; u = next;
    }
    if (u != start) return {};
    r.ok = true;
    return r;
}
} // namespace cp
```

M01 整数与位运算 Integer and Bits

识别：最大公约数、最小公倍数、二进制计数、取最低位、枚举 mask 的所有子集。

约定：gcd_abs 接受完整 ll 范围并返回 unsigned long long；gcd(0,0)=0。lcm_checked 只接受能放进非负 ll 的结果，否则 nullopt。位操作用 ull，位号 0..63；最低位不存在返回-1。

复杂度：gcd/lcm 为 O(log max(|a|,|b|))；64 位基础位操作 O(1)；子集枚举 O(2^popcount(mask))。

调用：ull g=gcd_abs(a,b); auto l=lcm_checked(a,b); if(l) cout<<*l; int cnt=bit_count(mask); int k=lowest_bit_index(mask);

易错：LLONG_MIN 不能直接取负或 abs；其绝对值 2^63 不能装进 ll。；使用 1ULL<<k 且 k<64，不要用有符号 1 处理高位。；子集枚举必须在 sub==0 时结束，否则会回到 mask。

例题：原仓库 gcd.cpp：https://github.com/ball-castle/template/blob/main/gcd.cpp 。旧递归在(-6,-4)返回-2；新版返回非负 gcd 并覆盖完整 64 位输入。

源码 math/integer_bits.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// 绝对值用无符号保存，避免 -LLONG_MIN 的有符号溢出。
inline ull unsigned_abs(ll x) {
    return x < 0 ? 0ULL - ull(x) : ull(x);
}
inline ull gcd_abs(ll a, ll b) {
    ull x = unsigned_abs(a), y = unsigned_abs(b);
    while (y) { ull r = x % y; x = y; y = r; }
    return x;  // gcd(0,0)=0；结果可能为 2^63，不能总转回 ll。
}
inline optional<ll> lcm_checked(ll a, ll b) {
    if (a == 0 || b == 0) return 0;
    ull x = unsigned_abs(a), y = unsigned_abs(b);
    __int128 z = (__int128)(x / gcd_abs(a, b)) * y;
    if (z > LLONG_MAX) return nullopt;
    return ll(z);
}

// 位编号从 0 开始；无符号运算可以安全处理第 63 位。
inline ull lowbit(ull x) { return x & (0ULL - x); }
inline int bit_count(ull x) { return popcount(x); }
inline int lowest_bit_index(ull x) {
    return x == 0 ? -1 : int(countr_zero(x));
}
// 第 k 位：1ULL<<k，要求 0<=k<64；不可写 1<<k 处理高位。
// 判断位：(mask>>k)&1ULL；设置位：mask|=(1ULL<<k)。
// 枚举全部子集(含 0)，顺序从大到小，O(2^popcount(mask))：
// for (ull sub=mask;; sub=(sub-1)&mask) {
//     ... 使用 sub ...
//     if (sub==0) break;  // 必须先终止，否则又回到 mask。
// }
```

M02 安全模运算与快速幂 Modular Power

识别：大整数模乘、非负整数快速幂、负数归一化，以及质数条件成立时求逆元。

约定：1<=mod<=LLONG_MAX，底数可为任意 ll；结果在[0,mod)。指数为非负 unsigned long long；0^0 按 1 处理后取模，mod=1 返回 0。

复杂度：归一化/模加/模乘 O(1)，快速幂 O(log exp)。

调用：ll y=mod_pow(a,e,mod); ll p=mod_mul(a,b,mod); 若确认 mod 为质数且 a 不为 0 模 mod，可用 mod_pow(a,mod-2,mod)求逆元。

易错：先转__int128 再乘；乘完再转类型已来不及。；费马逆元 a^(mod-2)只适用于 mod 为质数且 a%mod!=0。；不能把负指数直接转成 unsigned 传入；指数取模也需要额外数论前提。

例题：原仓库 ksm.cpp：https://github.com/ball-castle/template/blob/main/ksm.cpp 。旧(-2)^3 mod5 为-3、3^0 mod1 为 1；新版分别为 2 和 0。

源码 math/modular.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 1<=mod<=LLONG_MAX；返回[0,mod)。支持负数和 LLONG_MIN。
inline ll mod_norm(ll a, ll mod) {
    assert(mod >= 1);
    a %= mod;
    if (a < 0) a += mod;
    return a;
}
inline ll mod_add(ll a, ll b, ll mod) {
    return ll(((__int128)mod_norm(a, mod) + mod_norm(b, mod)) % mod);
}
inline ll mod_mul(ll a, ll b, ll mod) {
    return ll((__int128)mod_norm(a, mod) * mod_norm(b, mod) % mod);
}
// exp 是非负整数；0^0 定义为 1 再取模；模 1 时答案恒为 0。
inline ll mod_pow(ll a, unsigned long long exp, ll mod) {
    a = mod_norm(a, mod);
    ll ans = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = mod_mul(ans, a, mod);
        a = mod_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 费马逆元：仅当 mod 为质数且 a%mod!=0 时，inv=mod_pow(a,mod-2,mod)。
// 复合模数不能套此公式；负指数不能直接传入本函数。
```